algorithm correctness

mahmood

February 11, 2023

Definition 0.0.1. an algorithm is totally correct if it receives valid input, terminates, and always returns the correct output

the only way to prove the correctness of an algorithm over all possible inputs is by reasoning formally or mathematically about it.
generally, to prove an algorithm is correct we prove two things

1. the algorithm terminates

2. the algorithm returns the correct value for every input

we propose a statement that describes what the algorithm takes as input and what its supposed to output and then prove it

1 proving algorithm correctness by induction

here we'll be using mathematical induction

Example 1.0.1.

```
Function Sum-Array(arr, n):
    sum ← 0
    for i ← 1 to n do
        sum ← sum + A[i]
    return sum
```

Lemma 1. for each input that is an array of numbers A, the algorithm SUM-ARRAY computes the sum of the numbers

Proof. <u>proof the algorithm terminates</u>: the algorithm consists of a for loop that is bound by n, so it follows trivially that it must terminate
<u>proof of correctness</u> by perfect induction on the length of the input array
we propose the following loop invariant for the loop

$$sum = \sum_{j=1}^{i} A[j]$$

in this case, proving the proposed loop invariant would prove the algorithm correct

1. for $i = 1$, which is the base case, the algorithm returns A[1] which would be the only element in the array, so its obvious that its the correct output

2. the induction hypothesis, we assume after i iterations in the loop that $sum = \sum_{j=1}^{i} A[j]$

3. the induction step, during the $i + 1$'th iteration of the loop, we add $A[i + 1]$ to the variable sum, and so it follows from this and from the induction hypothesis that:

$$sum = \sum_{j=1}^{i} A[j] + A[i + 1] = \sum_{j=1}^{i+1} A[j]$$

the loop runs n times, by substituting n in the loop invariant, we'd get

$$sum = \sum_{j=1}^{n} A[j]$$

which means that after the loop is done, the variable sum would contain the sum of the elements in the array, which is what the function returns, which is the correct output
QED

□

2 correctness of recursive algorithms

the pattern for proofs of recursive algorithms is as follows:

1. base step of the recursion, we have to manually show that for each of the edge cases the output is correct

2. induction hypothesis, we assume that in each recurring call the output is correct

3. proof, we show that the output is correct for the main function call

Example 2.0.1. consider the following function

```
Algorithm: sum of array elements
Function RECURSIVE-SUM(A):
    if |A| = 1 then
        return A[1]
    A_l ← left half of A
    A_r ← right half of A
    s_l ← RECURSIVE-SUM(A_l)
    s_r ← RECURSIVE-SUM(A_r)
    return s_l + s_r
```

proof that the algorithm terminates:
assume that RECURSIVE-SUM runs on an array `A`, on each recurring call, the size of the input array `A` is halved, its obvious that at the end of every sequence of calls the size of the input array would arrive to 1 which is the edge case that would cause the function to terminate
proof of correctness:
we prove this by induction on the number of elements in the array

1. base case: for $|A| = 1$ which is the edge case, the algorithm returns $A[1]$, the only element in A, its obvious that the output is correct

2. induction hypothesis: we use perfect induction on the size of the input, we assume that the algorithm correctly computes the sum of the elements of any array whose size is smaller than `n`

3. <u>induction step</u>: let `A` be an arbitrary array of size `n`, the call to RECURSIVE-SUM(A) executes the calls RECURSIVE-SUM(A_l) and RECURSIVE-SUM(A_r), the size of the arrays A_l and A_r is smaller than n, so according to the previous step we know those two recurring calls do return the correct sum of the corresponding subarrays, and since we are returning the sum of those sums we know for sure we are returning the correct sum